

Common Table Expression (CTE) in SQL Server

SQL SERVER CTE

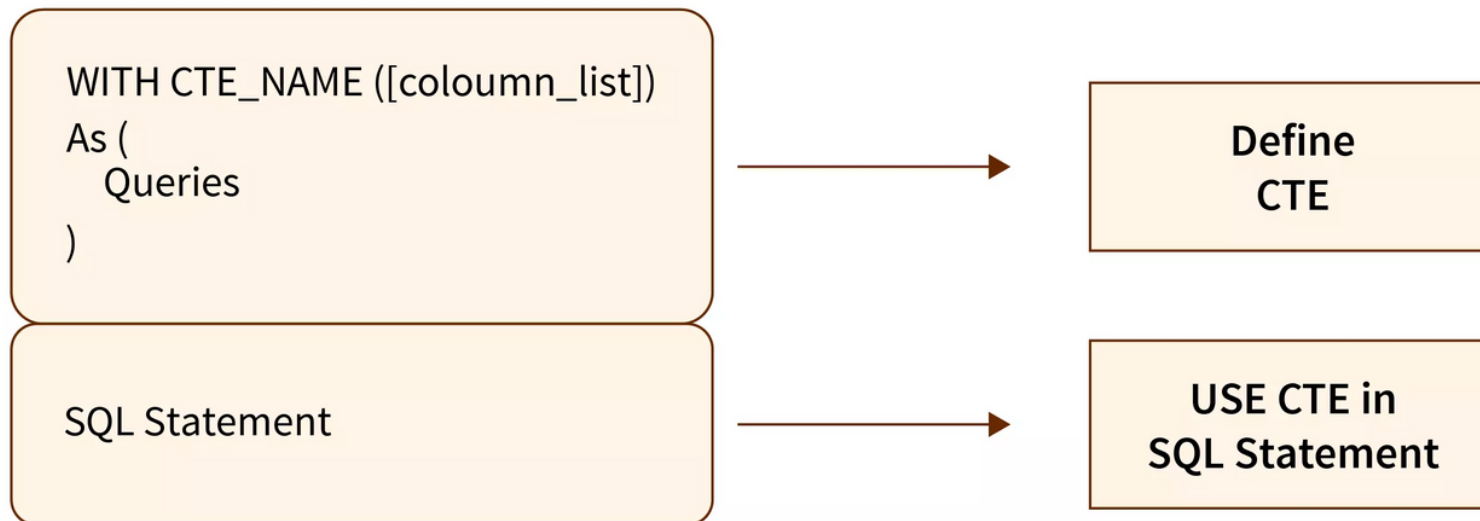
MS SQL

**With CTE_
AS
(**

?

What is CTE in SQL Server?

- A CTE (Common Table Expression) is a one-time result set that only exists for the duration of the query. It allows us to refer to data within a single SELECT, INSERT, UPDATE, DELETE, CREATE VIEW, or MERGE statement's execution scope.
- It is temporary because its result cannot be stored anywhere and will be lost as soon as a query's execution is completed.
- CTEs are typically the result of several sub-queries(nested queries to get some information needed in the main query).



CTE Syntax in SQL Server

- The CTE syntax includes a CTE name, an optional column list, and a statement/query that defines the common table expression (CTE). After defining the CTE, we can use it as a view in a SELECT, INSERT, UPDATE, DELETE, and MERGE query.
- The following is the basic syntax of CTE in SQL Server:

```
WITH cte_name (column_names)  
AS (query)  
SELECT * FROM cte_name;
```

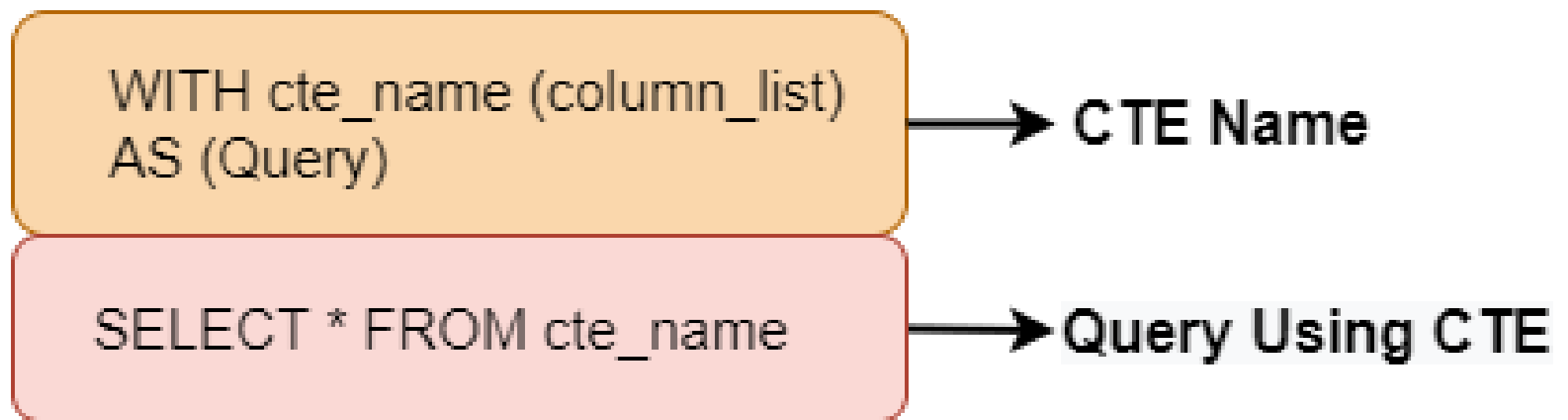
In this syntax:

- We have first specified the CTE name that will be referred to later in a query.
- The next step is to create a list of comma-separated columns. It ensures that the number of columns in the CTE definition arguments and the number of columns in the query must be the same. If we have not defined the CTE arguments' columns, it will use the query columns that define the CTE.
- After that, we'll use the AS keyword after the expression name and then define a SELECT statement whose result set populates the CTE.
- Finally, we will use the CTE name in a query such as SELECT, INSERT, UPDATE, DELETE, and MERGE statement.

It should keep in mind while writing the CTE query definition; we cannot use the following clauses:

1. ORDER BY unless you also use as TOP clause
2. INTO
3. OPTION clause with query hints
4. FOR BROWSE

The below image is the representation of the CTE query definition.



Here, the first part is a CTE expression that contains a SQL query that can be run independently in SQL. And the second part is the query that uses the CTE to display the result.

Example



Let us understand how CTE works in SQL Server using various examples. Here, we are going to use a table "customer" for a demonstration. Suppose this table contains the following data:

id	c_name	email	city
1	Steffen	stephen@javatpoint.com	Texas
2	Joseph	Joseph@javatpoint.com	Alaska
3	Peter	Peter@javatpoint.com	Califomia
4	Donald	donald@javatpoint.com	New York
5	Kevin	kevin@javatpoint.com	Florida
6	Marielia	Marielia@javatpoint.com	Arizona
7	Antonio	Antonio@javatpoint.com	New York
8	Diego	Diego@javatpoint.com	California
9	Thompson	thompson@javatpoint.com	New York
10	Charles	charles@javatpoint.com	Florida
11	Matthew	matthew@javatpoint.com	New York
12	Robert	robert@javatpoint.com	California

In this example, the CTE name is customers_in_newyork, the subquery that defines the CTE returns the three columns customer name, email, and state. As a result, the CTE customers_in_newyork will return all customers who live in New York State.



After defining the CTE customers_in_newyork, we have referenced it in the SELECT statement to get the details of those customers who are located in New York.

```
WITH customers_in_NewYork
AS (SELECT * FROM customer WHERE state = 'New York')
SELECT c_name, email, state FROM customers_in_NewYork;
```

After executing the above statement, it will give the following output. Here, we can see that the result returns only that customer information who are located in New York State.

c_name	email	state
Donald	donald@javatpoint.com	New York
Antonio	Antonio@javatpoint.com	New York
Thompson	thompson@javatpoint.com	New York
Matthew	matthew@javatpoint.com	New York



Multiple CTE

- In some cases, we'll need to create multiple CTE queries and join them together to see the results. We may use multiple CTEs concept in this scenario.
- We need to use the comma operator to create multiple CTE queries and merge them into a single statement. The "," comma operator must be preceded by the CTE name to distinguish multiple CTE.
- Multiple CTEs help us in simplifying complex queries that are eventually joined together. Each complex piece had its own CTE, which could then be referenced and joined outside the WITH clause.

The below syntax explains it more clearly:

WITH

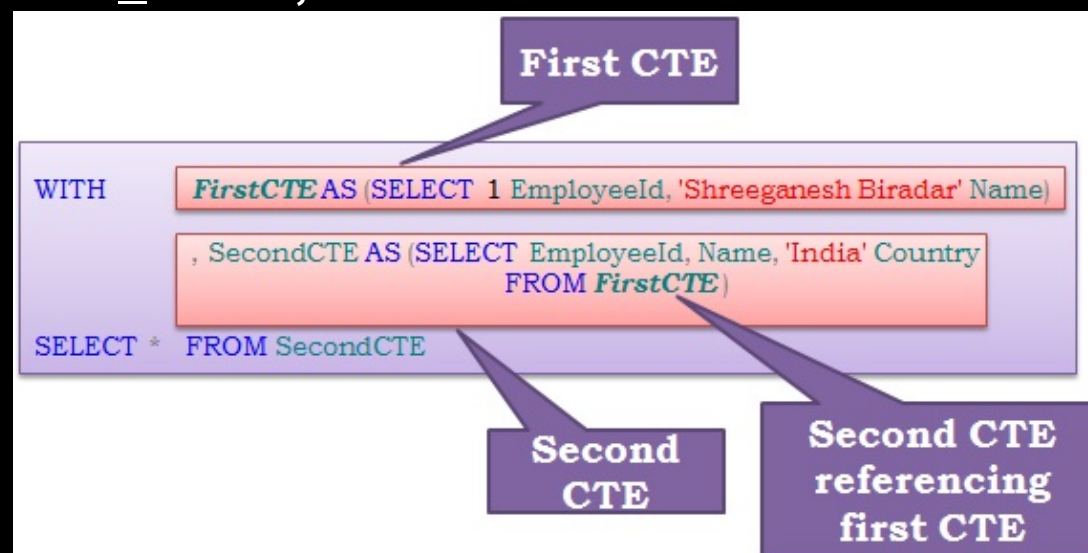
cte_name1 (column_names) AS (query),

cte_name2 (column_names) AS (query)

SELECT * FROM cte_name

UNION ALL

SELECT * FROM cte_name;



Example

Let us understand how multiple CTE works in SQL Server. Here, we are going to use the above "customer" table for a demonstration.

In this example, we have defined the two CTE names customers_in_newyork and customers_in_california. Then the result set of subqueries of these CTEs populates the CTE. Finally, we will use the CTE names in a query that will return all customers who are located in New York and California State.

WITH

customers_in_NewYork

AS (SELECT * FROM customer WHERE state = 'New York'),

customers_in_California

AS (SELECT * FROM customer WHERE state = 'California')

SELECT c_name, email, state FROM customers_in_NewYork

UNION ALL

SELECT c_name, email, state FROM customers_in_California;

c_name	email	state
Donald	donald@javatpoint.com	New York
Antonio	Antonio@javatpoint.com	New York
Thompson	thompson@javatpoint.com	New York
Matthew	matthew@javatpoint.com	New York
Peter	Peter@javatpoint.com	Califomia
Diego	Diego@javatpoint.com	Califomia
Robert	robert@javatpoint.com	Califomia



Why do we need CTE?

- Like database views and derived tables, CTEs can make it easier to write and manage complex queries by making them more readable and simple.
- We can accomplish this characteristic by breaking down the complex queries into simple blocks that can reuse in rewriting the query.

Some of its use cases are given below:

- It is useful when we need to define a derived table multiple times within a single query.
- It is useful when we need to create an alternative to a view in the database.
- It is useful when we need to perform the same calculation multiple times on multiple query components simultaneously.
- It is useful when we need to use ranking functions like ROW_NUMBER(), RANK(), and NTILE().

Types of CTE in SQL Server

SQL Server divides the CTE (Common Table Expressions) into two broad categories:

1. Recursive CTE
2. Non-Recursive CTE



Recursive CTE

- A common table expression is known as recursive CTE that references itself. Its concept is based on recursion, which is defined as "the application of a recursive process or definition repeatedly."
- When we execute a recursive query, it repeatedly iterates over a subset of the data. It is simply defined as a query that calls itself. There is an end condition at some point, so it does not call itself infinitely.
- A recursive CTE must have a UNION ALL statement and a second query definition that references the CTE itself in order to be recursive.

Example

Let us understand how recursive CTE works in SQL Server. Consider the below statement, which generates a series of the first five odd numbers:

```
WITH  
odd_num_cte (id, n) AS  
(  
  SELECT 1, 1  
  UNION ALL  
  SELECT id+1, n+2 from odd_num_cte where id < 5  
)  
SELECT * FROM odd_num_cte;
```



When we execute this recursive CTE, we will see the output as below:

id	n
1	1
2	3
3	5
4	7
5	9

Non-Recursive CTE

- A common table expression that doesn't reference itself is known as a non-recursive CTE. A non-recursive CTE is simple and easier to understand because it does not use the concept of recursion.
- According to the CTE Syntax, each CTE query will begin with a "With" clause followed by the CTE name and column list, then AS with parenthesis.



Advantages of CTE

Some of its advantages are given below:

- CTE facilitates code maintenance easier.
- CTE increases the readability of the code.
- It increases the performance of the query.
- CTE makes it possible to implement recursive queries easily.

Disadvantages of CTE

The following are the limitations of using CTE in SQL Server:

- CTE members are unable to use the keyword clauses like Distinct, Group By, Having, Top, Joins, etc.
- The CTE can only be referenced once by the Recursive member.
- We cannot use the table variables and CTEs as parameters in stored procedures.
- We already know that the CTE could be used in place of a view, but a CTE cannot be nested, while Views can.
- Since it's just a shortcut for a query or subquery, it can't be reused in another query.
- The number of columns in the CTE arguments and the number of columns in the query must be the same.



Conclusion

- Common Table Expressions (CTEs) are used to create a temporary relation by using the WITH clause in SQL.
- We can refer to the temporary result set created by the CTE in subsequent SQL Statements.
- CTE and subquery both are similar approaches but CTE provides several advantages like better code readability and easy maintenance.
- There are two types of CTE in SQL server: Recursive and Non-Recursive.
- Recursive CTE is specially used to work with the tree or hierarchically structured data.



Do You Find This Post Helpful?



**If Yes Then Share With Your
Network So That Everyone Get
Benefited**



Follow

